

Designing Consumer-Facing Agentic Interfaces

Executive summary

Consumer agentic interfaces are not simply chat interfaces with more capable models behind them. Once an AI can **act**, rather than merely answer, the UX problem changes from conversational design to **delegation, supervision, coordination, and recovery**. The central design challenge is no longer “How should the assistant respond?” It is “How does a person remain oriented and appropriately in control while another system performs work across time, tools, objects, and potentially consequential boundaries?” Recent agentic systems from OpenAI, Anthropic, Google, Perplexity, Replit, Manus, and Amazon are converging on visible planning, progress reporting, scoped permissions, takeover controls, confirmations for consequential actions, and increasingly explicit recovery mechanisms. ¹

The strongest design principle from the research is:

Design delegated work as a visible, inspectable transaction, not as magic happening behind a chat bubble.

At almost every point in an agentic task, the interface should make five questions cheap to answer:

User question	UI obligation
What is it going to do?	Goal, scope, constraints, plan or next action
What is it doing now?	Current state, active step, progress, touched objects
Why is it doing that?	Contextual rationale and source/provenance on demand
What did it change?	Outcome, affected objects, activity/change summary
What can I do about it?	Steer, pause, stop, take over, undo, retry, restore

This structure follows the direction of Google's People + AI guidance on mental models, explainability, control, and graceful failure; Microsoft's HAX work; IBM Carbon's AI provenance patterns; and newer agent-specific research such as Magentic-UI and AgentTrace. ²

The biggest mistake I would avoid is making that the information architecture. Chat is useful as an intent and steering surface, but a multi-step task should become an object with persistent state: goal, plan, constraints, resources, approvals, activity, outputs, and history. Magentic-UI explicitly explored editable plans, progress and action guards; Replit exposes plans, checkpoints and rollback history; AgentTrace's recent work argues for task timelines, resource touchpoints, authority history, provenance, and persistent-change summaries. ³

A second strong conclusion is that **autonomy should vary by consequence rather than by a single global “agent mode.”** Low-risk information retrieval can be highly autonomous. Reversible modifications can often execute with visible notification and undo. External communications, purchases, deletion, permissions, and other consequential side effects warrant stronger review or confirmation. OpenAI’s documented agent controls use confirmation before consequential actions and active supervision for some critical actions; Gemini Spark asks for confirmation for communications, data modification, purchases and web forms; Cowork can show a plan and wait before significant actions; Magentic-UI participants specifically preferred action guards for payments, sending emails, and subscriptions. 4

Conversely, **do not turn safety into a thousand tiny “Allow?” dialogs.** Confirming every harmless substep creates approval fatigue and trains users to click through. The better pattern is a **risk-adjusted commit boundary**: let the agent autonomously perform low-risk preparation, then stop exactly where an external or hard-to-reverse side effect begins. Replit’s separation between planning and building and its checkpoint model provide a particularly useful reference for this principle. 5

For proactivity, **timing matters as much as relevance.** A controlled 2025 study of proactive programming assistants found that 90% of participants preferred a contextual Suggest condition and 80% preferred Suggest + Preview, versus only 47% for persistent suggestions. A 2026 field study similarly found substantially higher engagement for proactive interventions at workflow boundaries than for mid-task interruptions. A 2026 IUI study found comparable task performance between on-demand and user-scheduled assistance, but more favorable perceptions of the pre-scheduled condition. These are not consumer-scale universal laws, but they all point in the same direction: proactivity should be contextual, interruptible, tunable, and sparse. 6

Recommended default architecture

For a general consumer agentic product, I would establish six levels of autonomy:

Level	Agent behavior	Default UX
Advise	Produces information or suggestions	Inline result; citations/evidence where relevant
Draft	Creates a proposed artifact or change	Editable preview; Apply/Discard
Stage	Prepares a multi-object action	Summary of effects; Review changes
Execute	Makes reversible changes	Execute; persistent status; Undo/history
Commit	Creates consequential external side effects	Explicit, object-specific confirmation
Background	Runs later or repeatedly	Persistent schedule/policy object; notifications; activity history; pause/revoke

This is a synthesis of patterns now appearing across OpenAI, Gemini Spark, Claude Cowork, Replit Agent and Alexa for Shopping, rather than a published industry standard. 7

The practical screen-design consequence is equally important: **show less reasoning, more state**. Users generally need the goal, plan, current step, material decisions, affected objects, evidence, and recovery controls. They do not need an endless textual play-by-play of every low-level tool operation. AgentTrace's work explicitly organizes raw execution into higher-level timelines and resource summaries, while Magentic-UI found that even progressively disclosed execution histories could become overwhelming. ⁸

Finally, the success metric for an agentic UI should not be engagement with the assistant. It should be **verified task success at acceptable supervisory cost**. A good agent UI should ultimately require *less* babysitting, not produce an exciting new genre of babysitting.

Evidence base and product landscape

What the literature actually supports

The evidence base for agentic UX is growing rapidly, but it is still immature. Recent work contains controlled studies, field studies and sophisticated research prototypes, yet much of the empirical literature is concentrated in programming, web automation, knowledge work and laboratory problem-solving rather than longitudinal mass-market consumer environments. Product documentation reveals strong convergent patterns, but vendors rarely publish controlled UX validation for individual UI choices. Accordingly, some recommendations below are high-confidence standards, others are evidence-backed design hypotheses that still deserve product-level testing. ⁹

Prioritized literature and design references

Priority	Source	Why it matters for agentic UI	Design implications
A, empirical	Magentic-UI, Microsoft Research, 2025	Human-in-the-loop agent research explicitly studies co-planning, editable plans, co-tasking and action guards. Participants endorsed guards particularly for high-risk actions such as payment, email and subscriptions. ¹⁰	Make the plan manipulable; stop at consequential boundaries; support steering and resumption.
A, empirical	Need Help? Designing Proactive AI Assistants, 2025	Controlled study showed strong preference for contextual Suggest and Suggest + Preview, while persistent suggestions performed materially worse on preference. ¹¹	Prefer sparse contextual suggestions over permanent nudging. Preview is especially useful when a suggestion could become an action.
A, empirical	AgentTrace, 2026	Develops and evaluates post-hoc traceability around task timeline, resource touches, permissions, provenance and persistent changes. ¹²	Treat audit/recovery as user-facing information architecture, not a developer log.

Priority	Source	Why it matters for agentic UI	Design implications
A, empirical	Mixed-Initiative AI, IUI 2026	Finds assistance delivery mode can affect perception independently of task performance; user-selected scheduled assistance was perceived more positively than purely on-demand help in the studied task. ¹³	Give people initiative preferences and timing controls rather than assuming a single “smart” interruption policy.
A, field evidence	Developer Interaction Patterns with Proactive AI, 2026	Five-day study with 15 developers and 229 interventions found receptivity varied strongly by workflow timing; boundary-timed interventions performed better than mid-task interventions. ¹⁴	Treat interruption timing as an interaction-design variable, not simply a model-confidence threshold.
A, living design guidance	Google People + AI Guidebook	Mental models, staged onboarding, explainability, calibrated trust, feedback/control and graceful failure are treated as connected interaction problems. ¹⁵	Explain capability and limits; expose control throughout use; design failure recovery before happy-path polish.
A, design system	IBM Carbon for AI	Provides explicit AI-presence labeling, progressive explainability and revert patterns. ¹⁶	Mark AI-created or AI-modified objects consistently; make provenance expandable; preserve revert paths after human edits.
A, accessibility standard	WCAG 2.2 / WAI	Defines contrast, target-size, status-message and interaction accessibility requirements relevant to dynamic agents. ¹⁷	Agent state, progress, errors and takeover controls must work for keyboard, screen-reader, low-vision and motor-impaired users.
B, foundational	Microsoft HAX Toolkit	Provides reusable patterns around common human-AI interaction problems. HAX predates the current wave of LLM agents, but its framework remains actively useful. ¹⁸	Use HAX as a heuristic layer, but pair it with newer agent-specific evidence.

Priority	Source	Why it matters for agentic UI	Design implications
B, UX lab	Nielsen Norman Group, AI design research, 2025	Reinforces that generated content still needs classic web-writing qualities such as concision, scanning and plain language; also reports risks from unnecessary anthropomorphism in task-oriented AI. ¹⁹	Do not let “AI” become an excuse for walls of prose or theatrical personalities.
B, platform guidance	Apple HIG, Generative AI	Apple's dedicated generative-AI guidance became part of its contemporary HIG and has continued to evolve. ²⁰	Keep generated experiences consistent with native interaction conventions; refinement and feedback belong inside the production workflow.
Frontier signal	CHIIR 2026 workshop on proactive/personalized agents	The workshop's synthesis emphasizes timing, transparency, contestability, autonomy, privacy and user welfare as central unresolved questions. ²¹	Treat proactivity and personalization as policy surfaces users should be able to understand and contest.

The literature also suggests a useful distinction between **process transparency** and **reasoning exposure**. Users benefit from seeing *what the system plans to do, what it has done, what resources it touched, and what decisions require attention*. That does not imply exposing raw model reasoning. AgentTrace deliberately aggregates execution into human-readable events; Magentic-UI exposes editable natural-language plans rather than hidden internal reasoning. ²²

Comparative analysis of leading agentic interfaces

The table below focuses on visible interaction patterns rather than model capability rankings.

Product / platform	Agent capabilities	Initiation	Control	Feedback	Error recovery	Trust signals
OpenAI ChatGPT agent lineage	Web navigation, research, tool use, connected data, code and multi-step actions. The July 2025 launch UX supported interruption, clarification, pause, takeover and scheduled tasks. OpenAI now labels that launch post outdated and points toward newer agentic work experiences, so treat these as pattern lineage rather than a frozen current UI spec. ²³	Composer/tool mode; natural-language task. Current help also documents agent initiation from the ChatGPT tool surface. ²⁴	Interrupt, steer, pause, stop and browser takeover. ²³	On-screen narration; progress summary; completion notification. ²³	Stop with partial results; intervene/take over. ²³	Confirmation before consequential actions; stronger active supervision for some critical actions; secure takeover for sensitive entry. ²³
Claude Cowork	Multi-step research, analysis, document creation, files, connected tools and computer use; web/mobile tasks can continue in the cloud. ²⁵	User defines a goal and chooses accessible files/tools. ²⁵	Redirect or stop; user chooses folders and tools; permissions can cause Claude to show a plan and wait for approval. ²⁶	Plan and ongoing Cowork session; task can be reviewed or redirected remotely. ²⁷	User can redirect/stop; destructive actions such as deletion require approval according to current Cowork documentation. ²⁶	Explicit resource scope, permission settings, significant-action approval. ²⁶

Product / platform	Agent capabilities	Initiation	Control	Feedback	Error recovery	Trust signals
Google Gemini Spark	Personal multi-step tasks, schedules, skills, connected apps, browser actions and recurring workflows. ²⁸	High-level task; task/schedule/skill become distinct building blocks. ²⁸	Stop response, browser takeover, scheduled-task management. ²⁸	Displays planning information, progress, relevant skills/apps/files used or created. ²⁹	Takeover for specific steps; confirmation provides links back to affected Workspace services for subsequent changes/undo. ³⁰	Confirmation for communication modifying data, purchases and forms; scope restrictions; takeover for sensitive information. ³⁰
Perplexity Comet Assistant / Agent	Contextual assistance across tabs plus more autonomous browser tasks such as scheduling, forms and multi-step workflows. ³¹	Persistent browser-side assistant; current page/tab context lowers invocation cost. ³²	Context bounded by browser and task; enterprise controls can govern domains and agent activity. ³³	Side-panel continuity and awareness across tabs. ³¹	Strongest documented recovery controls are less visible than Replit's checkpoint model; this is a relative weakness in the public UX documentation reviewed.	Agent checks before important or sensitive actions in current enterprise documentation; credentials are stored locally rather than on Perplexity servers according to its privacy FAQ. ³⁴
Manus Browser Operator	Browser automation inside authenticated services using the user's local sessions. ³⁵	Browser extension / agent request attached to the browsing environment. ³⁵	Local browser/session scope is part of the control model; Manus describes the operator as providing transparency and control. ³⁵	Execution happens against familiar websites, providing a visible environmental reference. ³⁵	Public materials emphasize task execution more strongly than an explicit cross-task rollback model, making recovery an area designers should examine critically rather than copy blindly.	Authenticated local sessions reduce the need to transplant credentials into separate agent environment; provenance still needs strong UI treatment. ³⁵

Product / platform	Agent capabilities	Initiation	Control	Feedback	Error recovery	Trust signals
Replit Agent	Plans, builds and modifies apps; tests its work; supports background work and multi-artifact creation. ³⁶	Mode selector distinguishes planning from building; structured task lists can precede implementation. ³⁷	User reviews task list before building; paid actions require confirmation. ³⁸	Plans, task progress and AI-generated checkpoint descriptions. ³⁹	Best-in-class pattern: automatic checkpoints at logical milestones, history timeline, inspect changes, rollback and even roll-forward. ⁴⁰	Preview/plan before change plus explicit recoverability provides observable evidence that experimentation is not necessarily destructive. ⁴¹
Amazon Alexa for Shopping	Conversational shopping, research, comparisons, cart construction, scheduled actions, price monitoring and some agentic purchasing. ⁴²	Dedicated chat, Amazon search bar, product/search-page AI surfaces and scheduled actions. ⁴²	Users can stage products in a cart, define conditions such as price targets, create schedules, or invoke Buy for Me. ⁴³	Product cards, comparisons, price history, cart, notifications and order status anchor conversation in concrete retail objects. ⁴²	Cart review and traditional order-management surfaces provide familiar recovery and inspection points. Buy for Me also routes resulting orders into order tracking. ⁴⁴	"Add to cart, then review/check out" separates low-friction autonomous preparation from purchase commitment; Buy for Me shows full checkout details before requesting purchase. ⁴⁵

What the product comparison says

Three patterns look especially mature.

First, agents are becoming object-aware rather than purely conversational. Spark distinguishes tasks, schedules and skills; Replit separates plans, tasks, checkpoints and project state; Amazon grounds agent behavior in products, carts, orders and price conditions. This is exactly why an OOUX model is valuable before drawing screens. ⁴⁶

Second, the best control is contextual control. "Stop" belongs beside a running task. "Take over" belongs beside browser execution. "Undo" belongs beside the completed change. "Approve" belongs beside the exact consequential action. A giant global "AI settings" page is necessary for policy but insufficient for moment-to-moment agency. OpenAI, Gemini and Cowork all put control into the active task loop. ⁴⁷

Third, recovery is still under-designed across the category. Replit's checkpoint/history model and AgentTrace's audit model are substantially more informative than a generic “Something went wrong, retry?” response. This is fertile design territory. ⁴⁸

Screen architecture and interaction model

Gestalt-level hierarchy

A durable agent-task screen should visually privilege **task state over conversation chronology**.

For complex delegated work, structure the screen around four perceptual regions:

Hierarchy	What belongs there	Why
Goal and scope	One-line goal, important constraints, destination/account, schedule	Keeps the contract visible
Plan and current state	Macro steps, active step, paused/waiting/working status	Preserves orientation
Attention surface	Clarification, approval, conflict or exception requiring the user	Separates “FYI” from “I need you”
Outcome and evidence	Result, affected objects, sources, changes, undo/history	Makes completion verifiable

This structure operationalizes Google PAIR's mental-model and control guidance and mirrors the movement toward editable plans, progress, action guards and traceability in Magentic-UI and AgentTrace. ⁴⁹

A desktop implementation can use a two-column workspace, with plan/context on one side and active execution/output on the other. Mobile should **not** attempt to miniaturize the desktop workspace. Stack the same objects into a task view: persistent task status, expandable plan, active attention card, output, then activity/history. WCAG's mobile guidance applies the same relevant accessibility criteria across mobile interfaces. ⁵⁰

Information density and progressive disclosure

Use three disclosure levels:

Level one: glanceable state. Show `Working`, `Waiting for you`, `Paused`, `Verifying`, `Done`, `Needs attention`, or `Failed`, plus the active macro-step.

Level two: human-readable task detail. Show the editable plan, affected resources, assumptions, important decisions and meaningful milestones.

Level three: forensic detail. Provide activity history, raw tool calls where useful, timestamps, permissions, provenance and before/after state.

The ordering matters. AgentTrace's research specifically converts lower-level operations into high-level timeline events and provides expandable detail, while its resource map aggregates touched files, domains, credentials and other resources rather than presenting an undifferentiated log. ¹²

Do not continuously stream every thought-shaped sentence the system can produce. Process transparency is useful; narrative noise is not. Keep the primary surface focused on things that change user decisions.

A reasonable design-system starting point for a complex task is **one goal, approximately three to seven macro plan steps visible at once, one current attention item, and one clearly emphasized next user action**. That numeric range is a design heuristic, not a research-derived threshold. Test it against actual task complexity.

Modal versus inline behavior

Situation	Preferred container	Rationale
AI suggestion about an object already on screen	Inline card / anchored popover	Maintains context; low interruption
Generated field or object edit	Inline preview/diff	Allows immediate comparison and revision
Routine progress	Inline status region	No need to steal focus
Non-blocking warning	Inline banner/status	Keeps work moving
Clarification that blocks the current task	Inline attention card or mobile sheet	The question belongs to the task state
External send, payment, deletion or consequential commit	Modal/dialog/sheet	Deliberate interruption is justified
Long multi-step delegation	Dedicated task workspace	Chat chronology is too weak an IA
Ambient assistance while browsing	Side panel	Maintains relationship to underlying content, as Comet demonstrates. ³¹
Completed reversible action	Inline result plus Undo	Recovery should remain close to the outcome

The principle is simple: **modal interruption should correlate with decision consequence, not with AI uncertainty alone**. An uncertain recommendation can often be inline. A \$900 purchase should not be.

Risk-adjusted control model

A useful control policy is:

Action class	Example	Default autonomy	UI guard
Read-only	Search, summarize, compare	High	Source/resource disclosure
Local reversible	Reorder items, draft text, modify working copy	Medium-high	Change preview or persistent Undo
External but recoverable	Reschedule event, add to cart	Medium	Summary plus post-action recovery
External communication	Send message, invite attendee	Lower	Review recipients/content; explicit Send
Financial / legal / account consequence	Purchase, subscription, disclosure, permissions	Low at commit	Explicit object-specific confirmation
Destructive / hard to reverse	Delete records, cancel booking with penalty	Low	Consequence statement and explicit destructive action
Background / recurring	Auto-buy, recurring report, inbox cleanup	High during approved scope	Persistent policy/schedule object, notifications and easy suspension

This model is consistent with the action-guard preferences found in Magentic-UI and with current confirmation policies documented by OpenAI and Google. ⁵¹

Interaction patterns by initiative

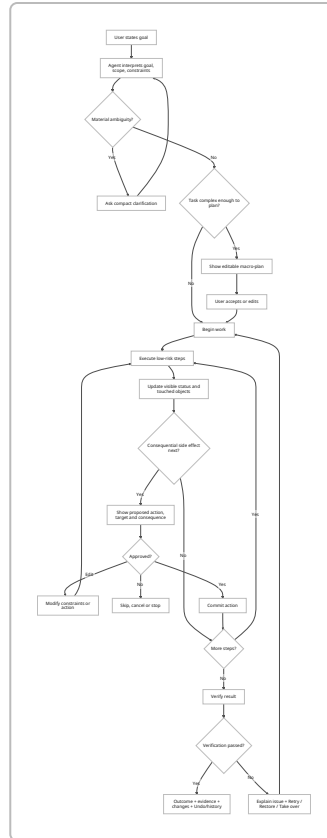
Moment	Pattern	Recommended behavior
User initiation	Intent + scope	Let users express a goal naturally; turn a sufficiently complex request into a persistent Task object. Surface inferred constraints for correction.
Agent suggestion	Contextual proposal	Show what the agent noticed, what it proposes, expected effect, <code>Review</code> / <code>Apply</code> , <code>Dismiss</code> , and optional <code>Why this?</code> .
Agent interruption	Attention request	Interrupt only for blocking ambiguity, elevated risk or material conflict. Prefer workflow boundaries and user-configurable timing. Proactive-assistant studies show persistent or poorly timed interventions can substantially reduce preference or engagement. ⁵²
Agent → user handoff	Takeover	Preserve task context and clearly state what the user must do, such as authenticate, choose between ambiguous options, or enter sensitive information. Google Spark and OpenAI both use browser takeover for sensitive or user-controlled steps. ⁵³

Moment	Pattern	Recommended behavior
User → agent handback	Resume	After takeover, explicitly indicate what changed and where the agent will resume.
User interruption	Steering	Pause execution while preserving progress; accept an amended constraint or plan; resume without forcing a restart. OpenAI documented this collaborative interruption model in its agent launch experience. ²³
Multi-turn work	Shared task state	Make plans, objects and constraints referenceable. “Change step 3” is better than making users paraphrase a 40-message conversation.
Completion	Verify then close	Show outcome, material changes, exceptions, evidence and next recovery action.
Background work	Notification + persistent task	Do not rely on notification alone. The notification should deep-link to a durable task record containing outcome and history.

Research supports making initiative at least partly user-configurable. The IUI 2026 mixed-initiative study found that how assistance was delivered changed user perception even without a task-performance difference, while the latest CHIIR workshop synthesis describes proactive behavior as needing to be timely, transparent and contestable. ⁵⁴

Canonical execution flow

This flow combines co-planning, risk-adjusted action guards, traceability and recovery into a single reusable interaction model. ⁵⁵



The key distinction is between **preparation** and **commit**. Preparation should generally optimize speed. Commit should optimize understanding and agency.

Trade-off map

Conceptual trade-off charts for autonomy, control, friction, and speed

[Download the trade-off diagram](#)

This chart is a **design heuristic, not measured quantitative data**. Its important implication is that reducing friction indiscriminately is the wrong optimization target. Consumer agent UX needs *selective friction*. The right click at the right boundary can prevent far more cost than it creates. Magentic-UI's participants differentiated high-risk actions such as payment and email from lower-risk steps, while current production agents similarly employ consequential-action confirmations. ⁵¹

Detailed UI system rules

Actions, buttons and affordances

Use **verb + object + consequence** whenever the action has material effects.

Weak	Better
Continue	Review changes
Yes	Send 3 messages
Confirm	Move appointment to 2:30 PM
Proceed	Buy for \$428.16
Allow	Allow calendar access for this task
Delete	Delete 12 saved searches
Done	Keep changes
Try again	Retry failed step
Cancel when execution has started	Stop task
Back after agent modifications	Discard draft changes

Consequential confirmation should repeat the **target object**, not merely the verb. “Send” is insufficient when an agent has selected recipients autonomously. Show who, what, where, amount, timing and relevant consequence.

For destructive decisions, separate the destructive button from routine navigation, provide clear consequence copy, and never rely on red alone to signal danger. WCAG explicitly requires information and states not to depend only on hue, and user-interface boundaries/states need sufficient non-text contrast.

56

For mobile and pointer environments, WCAG 2.2 Level AA establishes a 24 × 24 CSS-pixel target-size minimum or specified spacing exceptions; W3C recommends considering the stronger enhanced target-size guidance for important controls. Agentic , , , approval and destructive controls deserve generous targets because accidental activation can alter task state. ⁵⁷

My implementation baseline would therefore use **44–48 px effective target height for high-value agent controls**, even where the visual icon itself is smaller. That is deliberately more generous than the WCAG AA minimum.

Primary versus secondary action logic

Use one obvious primary action per decision surface.

For a **suggestion**: is primary; is secondary.

For a **plan**: is primary; and are secondary.

For a **consequential approval**: the committed verb is primary only after the consequences are visible; or remains immediately available.

For a **failure**: the action most likely to restore task progress should be primary, often `Retry step`, `Restore checkpoint`, or `Take over`. Do not automatically make `Start over` the default. Google PAIR explicitly treats the ability to move forward after AI failure as a central design question. ⁵⁸

During live execution, **Stop** should not disappear into an overflow menu. Neither should a required `Take control` action. Visibility is part of the user's mental model that the agent remains subordinate to them. OpenAI, Gemini Spark and Cowork all document interruption, stop or takeover behavior as explicit agent controls. ⁵⁹

Status vocabulary

Standardize your agent state machine and its language. I recommend:

Internal state	User-facing label	Supporting microcopy
<code>interpreting</code>	Planning	"Checking the request and constraints..."
<code>blocked_ambiguity</code>	Waiting for you	"I need one detail before I continue."
<code>executing</code>	Working	"Finding available appointment times..."
<code>approval_required</code>	Needs approval	"Review this change before it is sent."
<code>paused_user</code>	Paused	"No further actions will run until you resume."
<code>takeover</code>	Your turn	"Enter payment details in the browser, then hand control back."
<code>verifying</code>	Verifying	"Checking that all three calendar changes were saved."
<code>complete</code>	Done	"3 events updated; 5 attendees notified."
<code>partial</code>	Partially complete	"2 of 3 changes succeeded."
<code>failed_recoverable</code>	Needs attention	"The airline session expired. Sign in again or skip this step."
<code>failed_terminal</code>	Couldn't complete	"No reservation was made."

Avoid state copy such as "Thinking hard...", "I'm nervous about...", or other pseudo-emotional theater. Google PAIR warns that human-like framing can create unrealistic expectations, and recent NN/g research reports that task-oriented trust benefits more from perceived competence than apparent sentience. ⁶⁰

Clarification microcopy

A clarification should answer three things:

What is missing?

"Which airport should I use?"

Why does it matter?

"There are two airports within your requested distance and the fares differ significantly."

What are the easiest valid answers?

O'Hare · Midway · No preference

Do not ask the user questions the system can resolve safely from existing context. But do ask when assumptions materially change money, recipients, timing, privacy, irreversible outcomes, or the user's stated objective. Magentic-UI explicitly treats directed clarifying questions as part of co-planning. ⁶¹

When several tightly related clarifications are already known, consider **one compact grouped interruption** rather than repeatedly stopping after each answer. The goal is not zero questions. The goal is the highest useful information gain per interruption.

Confirmation and undo

Use **confirmation before hard-to-reverse commitment** and **undo after easy-to-reverse commitment**.

That difference is crucial.

Add 8 products to cart can usually execute and then expose review/removal. Amazon's agentic shopping experience demonstrates this staging pattern by adding products for subsequent review and checkout. ⁶²

Charge card \$428 deserves confirmation.

Move a card in a private board may be better served by an Undo affordance.

Email 117 customers deserves explicit recipient-and-content review.

Generate a draft email does not.

When rollback can restore an entire state, expose it as a durable object, not a transient snackbar. Replit's automatic checkpoints are created at meaningful milestones and support timeline navigation, rollback and roll-forward, a powerful reference architecture for reversible agent action. ⁶³

Visual feedback and animation

Use animation to convey **state transition and causality**, not "AI magic."

Good uses include a progress indicator moving to the next macro step, an updated object visibly transitioning from Draft to Applied, an approval card collapsing into the completed activity timeline, or a takeover state shifting control from agent to user.

Avoid continuous sparkle, pulsing gradients, orbital particles, fake typing when actual task state is known, and decorative motion during long-running work. Such animation competes with the information that actually matters: state.

Treat `prefers-reduced-motion` as a first-class requirement. W3C documents it as a technique for suppressing non-essential interaction animation for users who can experience distraction or vestibular symptoms. ⁶⁴

For implementation, a sensible starting token is approximately **120–220 ms for small state transitions** and somewhat longer only when motion is genuinely explaining spatial movement. This is a design starting point, not a WCAG requirement.

Accessibility for dynamic agents

Agent interfaces create unusual accessibility challenges because significant changes can happen **without direct user input at the instant they occur**.

Do not constantly steal focus as the agent works. Instead, expose routine progress through programmatically identifiable status messaging. WCAG 4.1.3 specifically covers success, result, waiting, progress and error information that appears without a change of context; WAI documents `role="status"` as one technique for politely notifying assistive technology. ⁶⁵

Use assertive announcement only for truly urgent attention. Errors can be exposed through appropriate live-region techniques without forcing focus movement. ⁶⁶

A good accessibility state model should ensure:

Agent event	Accessible treatment
Routine progress	Polite status announcement
Step completed	Short status update, not focus jump
Agent asks clarification	Move focus only when the user must act and context warrants it
Approval required	Dialog/sheet with clear accessible name and consequence
Task completes in background	Notification plus durable task state
Error	Text explanation; programmatically exposed error; recovery action
Visual diff	Equivalent textual “changed from / changed to” representation
Agent animation	Reduced-motion alternative
Icons/status colors	Text labels; never color alone
Expandable activity log	Keyboard operable; logical heading/focus structure

Typography and content

AI output should obey normal information-design rules. NN/g's current AI study guide explicitly notes that generated content still needs to be concise, scannable, plain-language and written for the web. ⁶⁷

Recommended starter tokens across responsive layouts:

Role	Starting point	Guidance
Screen/task title	24–32 px	Describe the user's objective, not “AI Assistant”
Section heading	18–22 px	Goal, Plan, Activity, Result
Body	16–18 px	Default for conversational and task content
Button/label	14–16 px	Keep consequential labels highly legible
Metadata	12–14 px	Time, source, tool, activity details only
Code/IDs	12–14 px monospace	Reserve monospace for genuinely technical objects
Body line-height	~1.45–1.6	Starting range; validate per typeface

Prefer sentence case. Do not use tiny gray text for the very information users need to decide whether to authorize an action.

Long outputs should lead with the outcome, then evidence, then optional detail. That is classic inverted-pyramid information design applied to generative content, consistent with NN/g's guidance. ⁶⁷

Color and contrast

Color should encode hierarchy **after** state has already been communicated by text, iconography or structure.

WCAG's minimum contrast for ordinary text is 4.5:1, with 3:1 applying to qualifying large text; user-interface components and meaningful non-text graphics require appropriate non-text contrast, commonly 3:1 against adjacent colors under the criterion. ⁶⁸

The following neutral example palette was checked using the WCAG relative-luminance contrast calculation:

Role	Foreground	Background	Calculated contrast	Usage
Primary text	#111827	#FFFFFF	17.74:1	Body/headings
Secondary text	#4B5563	#FFFFFF	7.56:1	Metadata
Primary action	#1D4ED8	#FFFFFF	6.70:1	Button fill with white label or blue text

Role	Foreground	Background	Calculated contrast	Usage
AI/provenance accent	#6D28D9	#FFFFFF	7.10:1	Optional AI indicator
Error/destructive text	#B42318	#FFFFFF	6.57:1	Error/destructive label
Success text	#047857	#FFFFFF	5.48:1	Completed state

These are illustrative tokens, not a prescription that agents should be blue, purple, or anything else. An AI gradient is not a trust model.

IBM Carbon's stronger underlying idea is useful: a **consistent AI marker can serve as an entry point into explainability**. Carbon's AI label marks AI-present/generated components and opens a summarized explanation; Carbon also supports reverting AI-suggested values after manual editing. ¹⁶

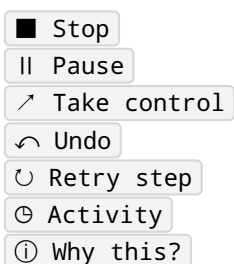
Imagery and iconography

Prefer **object evidence over AI decoration**.

For a shopping task, show product imagery and price. For a calendar action, show the actual event and attendees. For a document edit, show the changed paragraph or diff. For a travel task, show the itinerary segment being acted on.

A generic robot portrait tells the user virtually nothing about whether the agent is about to reschedule Grandma's cardiologist appointment.

For critical controls, pair icons with text wherever room permits:



Do not make the sparkle icon carry all semantic responsibility for "AI generated," "agent running," "suggestion," "automation" and "magic." Those are different states.

Spacing and grouping

A practical responsive token scale is:

4, 8, 12, 16, 24, 32, 48

Use tight spacing within one object, larger spacing between separate task concepts, and containment only when it communicates an actual object or state boundary.

For example:

- 8–12 px inside a status row.
- 12–16 px between fields of one approval object.
- 24 px between Plan and Activity.
- 32–48 px between major screen regions on larger displays.

This is a starter system, not an accessibility standard. The Gestalt goal is to make **Goal, Plan, Attention and Result** perceptually distinct without turning every sentence into its own floating card.

Localization

Agentic interfaces have two localization layers: the application UI and the generated content.

Design button containers, clarification choices, status pills and approval sheets to grow with translated text instead of assuming English width. W3C's internationalization guidance specifically recommends avoiding highly constrained spaces and allowing containers/backgrounds to expand with translated text. ⁶⁹

Also design for:

- RTL mirroring and correct directional metadata.
- Local time zones, especially for scheduled/background actions.
- Localized currency and number formats before monetary confirmation.
- Locale-specific dates, but include unambiguous dates where mistakes would matter.
- Pluralization rather than concatenated strings such as `Send + 3 + emails`.
- Grammar that allows dynamic object names to move within the sentence.
- Proper language metadata for agent-generated text that may switch language.

Current W3C work continues to emphasize explicit language and direction metadata for web strings. ⁷⁰

OOUX model and screen mapping

The strongest architectural recommendation in this report is to treat **Task** as the central user-facing object, not **Message**.

Messages are events in the task. They are not the task itself.

Core object model

Object	Core attributes / states	Relationships	Important user verbs	Primary UI mapping
Task	goal, status, owner, created, risk tier, completion	contains Plan, Constraints, Events, Outputs	create, delegate, pause, resume, stop, inspect, archive	Task card, task workspace
Goal	desired outcome, success criteria	belongs to Task	define, revise, verify	Header / objective editor
Constraint	budget, time, preferences, prohibited actions	limits Task/Actions	add, edit, remove, lock	Chips, constraints panel
Plan	ordered macro steps, revision, status	belongs to Task; contains Steps	review, edit, reorder, approve	Plan panel
Step	label, state, dependency, assigned agent/tool	belongs to Plan; contains Actions	skip, retry, edit, inspect	Progress list
Action	verb, target, side effect, risk, state	targets Resource; may require Approval	approve, deny, retry, undo	Activity row / approval card
Resource	file, site, account, product, calendar, message, person	touched by Actions	view, permit, revoke, inspect	Object chip/card
Approval	proposed action, scope, consequence, decision	guards an Action	approve, edit, deny	Confirmation dialog/sheet
Artifact / Outcome	document, itinerary, cart, modified record, result	produced/ modified by Task	open, edit, share, revert	Result view
Evidence	source, verification result, before/after state	supports Action or Outcome	inspect, compare	Explainability/ detail popover
Event	timestamp, actor, action, result	chronological Task history	inspect, filter	Activity timeline
Permission policy	resource scope, allowed verbs, duration	applies to Agent/ Actions	allow, revoke, narrow	Settings + contextual permission UI

Object	Core attributes / states	Relationships	Important user verbs	Primary UI mapping
Schedule	trigger, cadence, next run, expiration	invokes Task	enable, pause, edit, delete	Schedule object
Memory / preference	fact, source, scope, expiry	informs Tasks	review, correct, forget	Memory/ preferences screen

This object model reflects patterns appearing in contemporary agent platforms, particularly Gemini Spark's separation of task/schedule/skill, Replit's explicit plan/checkpoint/history objects, and the resource/action/authority distinctions explored by AgentTrace. ⁷¹

Verb architecture

The user's high-value verbs should be first-class product capabilities:

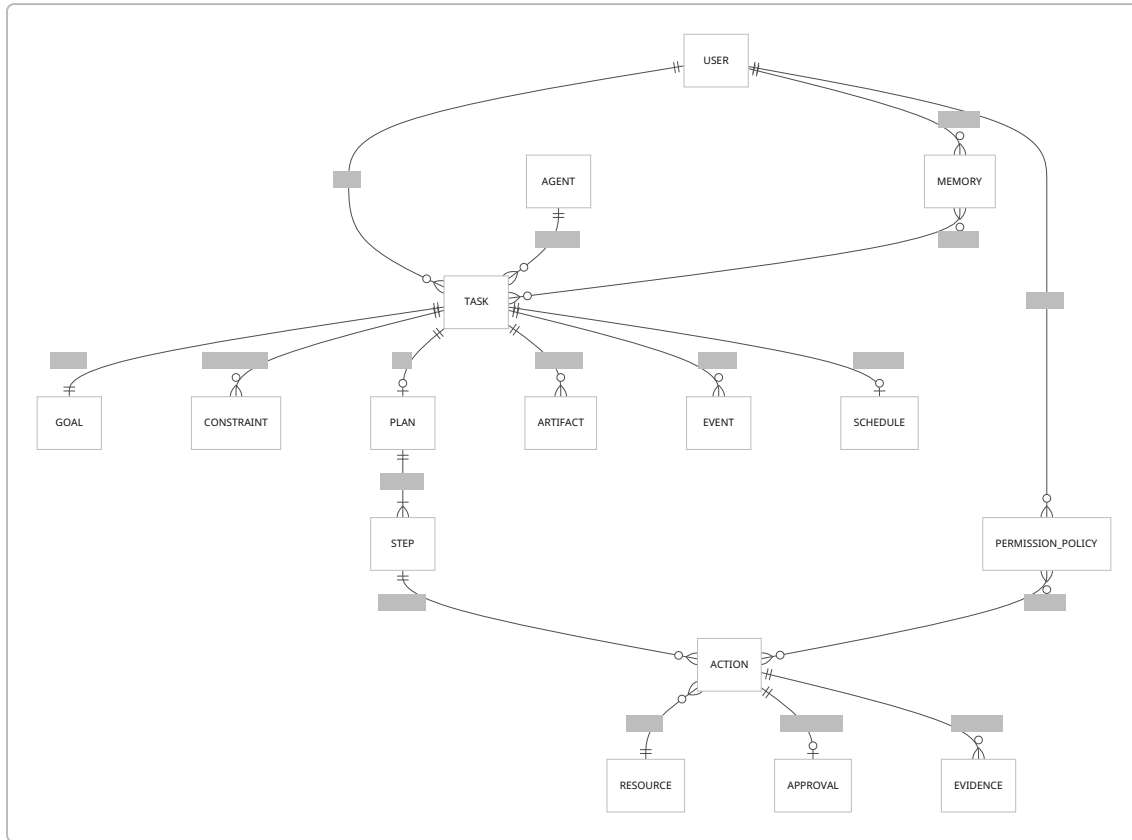
Delegate → Clarify → Review → Approve → Steer → Pause → Resume → Take over → Inspect → Verify → Undo → Retry → Restore → Schedule → Revoke.

The agent's system verbs are different:

Interpret → Propose → Plan → Read → Create → Modify → Communicate → Request approval → Execute → Observe → Verify → Report → Recover.

Separating these vocabularies is useful for the AI model assisting with flows. It prevents a common design failure where the model generates screens around whatever phrases occurred in the conversation rather than around durable actions and objects.

Entity relationship model



AgentTrace independently arrives at a particularly useful execution-oriented model involving actions, resources, authorities, triggers and persistent deltas. Its user-facing prototype then maps those entities into a timeline, resource touch map, permission history, provenance inspector and persistent change summary. ¹²

Object-to-screen mapping

Do not make every object a screen. Map screens to clusters of objects and high-frequency verbs.

Screen	Primary objects	Primary verbs	Components
Agent home / task inbox	Task, Schedule, Suggestion	start, resume, dismiss, pause	Task cards, suggested-action card, composer
Task workspace	Goal, Plan, Step, Constraint	steer, edit plan, pause, stop	Goal header, plan, progress, context chips
Attention center	Approval, Clarification, Conflict	approve, deny, answer, edit	Attention cards / modal sheets
Activity / audit	Event, Action, Resource, Evidence	inspect, filter, compare	Timeline, touched-resource list

Screen	Primary objects	Primary verbs	Components
Result / artifact	Artifact, Outcome, Evidence	open, edit, share, revert	Native artifact view plus provenance
Recovery	Checkpoint, Action, Artifact	undo, restore, retry	History, diff, checkpoint picker
Schedules / automations	Schedule, Task, Constraint	enable, pause, change cadence, revoke	Automation list/detail
Permissions and memory	Permission policy, Memory, Resource	narrow, revoke, correct, forget	Scope matrix, memory list

This mapping gives a screen-generating AI much firmer scaffolding than “make an AI assistant page.”

A useful prompt contract for that design model is:

For every screen, identify the primary object, its state, the user's available verbs, agent-generated state transitions, consequential actions, and the nearest recovery path before generating components.

That rule alone will prevent a lot of shiny nonsense.

Annotated scenarios and design flows

Proactive suggestion

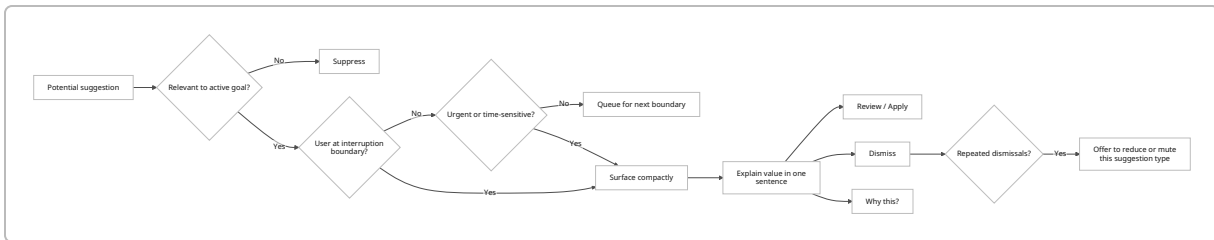
Annotated proactive agent suggestion wireframe

[Download the proactive-suggestion SVG](#)

The important pattern is that the agent **suggests near the triggering object rather than hijacking the whole experience**. The proposal identifies the contextual reason, stages the action behind **Review** **change**, provides **Dismiss**, and makes “Why this?” available through progressive disclosure.

This is strongly aligned with the proactive-assistant evidence. In the 2025 controlled study, 90% of participants preferred the Suggest condition and 80% preferred Suggest + Preview, compared with 47% for persistent suggestions. The finding is domain-specific, but the direction is persuasive: *timely value beats omnipresent eagerness*. ¹¹

A proactive-suggestion decision model should consider:



The 2026 field study strengthens the timing argument: proactive interventions at workflow boundaries achieved much stronger engagement than several mid-task contexts, showing that receptivity is coupled to task phase. ¹⁴

Multi-step task automation

Annotated multi-step task automation wireframe

[Download the task-automation SVG](#)

This is the canonical agent workspace I would build first.

The top contains the **delegation contract**: goal and constraints.

The left column is a **macro-plan**, editable because planning is a shared object rather than a transcript.

The right column shows **current execution**, resources touched, and whether side effects have occurred.

The approval card appears only when the agent reaches an external action.

The bottom checkpoint gives recovery persistent real estate instead of hiding it behind an error state.

That pattern combines Magentic-UI's editable co-planning and action guards with AgentTrace's resource/action transparency and Replit's checkpoint model. ⁷²

The most important microcopy in the wireframe is arguably:

No external changes made yet

That one sentence answers an extremely important user question: "Has the agent crossed the boundary from investigation into consequence?"

More interfaces should say this kind of thing.

Clarification dialog

Annotated mobile clarification dialog wireframe

[Download the clarification-dialog SVG](#)

Clarification needs to be cheap enough that users do not resent being asked, but meaningful enough to avoid an expensive wrong turn.

This design deliberately avoids the lazy agent pattern:

“Could you please provide more details?”

Instead it asks a **decision-specific question**, offers viable answers, explains why the distinction matters, and preserves a free-form escape hatch.

The clarification should pause only the dependent branch of work where architecture permits. For parallelizable tasks, other safe work can continue. This becomes increasingly important as multi-agent and background execution mature.

Handoff pattern

For browser or tool takeover, design the handoff as an explicit state transition:

Agent control

Working: opening checkout

↓

Handoff requested

Your turn: enter the card security code. I won't receive what you type.

Take control

↓

User control

Persistent banner: You are controlling the browser

Hand back to agent

↓

Reconciliation

You completed payment verification. I'll continue with order review.

Resume task

Current Gemini Spark documentation explicitly uses takeover for passwords and payment information, and OpenAI's earlier agent interface similarly documented secure browser takeover. ⁵³

Do not silently infer that the user is “done” and resume autonomous clicking. Require an explicit handback or a clearly communicated, safely detectable completion condition.

Background task pattern

Background execution should have three separate surfaces:

At creation:

What will run, when, against which resources, with what limits.

During life:

Next run, last run, state, pause, modify and revoke.

After each run:

What happened, what changed, exceptions, and recovery.

Amazon's current Alexa for Shopping exposes recurring Scheduled Actions, while Gemini Spark separately models schedules from tasks. These provide useful precedents for treating recurring autonomy as a durable object rather than merely another conversational message. 73

A schedule should therefore read something like:

Restock dog food monthly
Runs: first Monday, 9:00 AM
Condition: only if last purchase was > 24 days ago
Maximum price: \$54
Action: add to cart, do not purchase
Next run: September 7

Edit · Pause · Run now

That is far safer and more intelligible than "Sure, I'll remember."

Implementation roadmap, checklist, and measurement

Prioritized implementation roadmap

Priority	Build	Why first	Minimum design artifacts
Foundation	Formal Task, Goal, Constraint, Step, Action and Outcome objects	Without explicit state, every downstream screen becomes ad-hoc chat UI	Object map; state machine; verbs; risk taxonomy
Visible delegation	Goal/constraints header, plan, current step, status vocabulary	Establishes the user's mental model before adding autonomy	Task workspace; responsive mobile state; empty/loading/paused states

Priority	Build	Why first	Minimum design artifacts
Control boundaries	Pause, Stop, Take over, steer/edit plan, risk-based approvals	Core agency and safety	Control hierarchy; confirmation patterns; permission model
Recovery	Undo, Retry step, activity timeline, before/after diffs, checkpoints where feasible	Agents will fail. A product without recovery is a demo wearing a blazer.	Recovery matrix; event model; failure taxonomy
Traceability	Touched resources, provenance, permission history, persistent change summary	Becomes essential as work spans systems and runs unattended	Activity/audit view; AI-label rules; provenance components
Proactivity	Suggestions, timing rules, mute/snooze, initiative preferences	Only add after users trust reactive delegation	Proactivity policy; interruption model; preference controls
Background autonomy	Schedules, conditional execution, notification, run history	Highest leverage, but also highest “what happened while I was gone?” risk	Schedule object; per-run history; limits/policies
Adaptive autonomy	Optional user-configured defaults by action/risk type	Mature stage; dangerous to personalize invisibly	Explicit autonomy preferences; audit; easy reset

The ordering is deliberate. **Recovery should precede aggressive proactivity.** You should not increase how often an agent acts until you can tell users clearly what it did and reliably get them back to safety. AgentTrace and Replit provide strong evidence for treating trace/history/recovery as foundational infrastructure rather than polish. ⁷⁴

A/B and product-research agenda

Do not optimize only for completion speed. An agent can be fast and terrifying.

Experiment	Hypothesis	Primary metric	Guardrails
Editable plan vs. hidden/no plan	Plans improve alignment for complex tasks but may add needless friction for simple ones	Verified task success; plan edits; time to start	Abandonment; perceived effort
Plan always vs. complexity-triggered plan	Plan screens are valuable above a complexity threshold	Completion + supervisory time	Time-to-action

Experiment	Hypothesis	Primary metric	Guardrails
Per-step approval vs. commit-boundary approval	Risk-bundled approvals reduce fatigue without increasing errors	Approvals per successful task	Unintended side-effect rate
Suggestion vs. Suggest + Preview	Preview improves acceptance where side effects are imaginable	Accepted suggestions / completed actions	Dismissal and regret/undo
Workflow-boundary suggestion vs. immediate suggestion	Boundary timing reduces interruption cost	Suggestion acceptance; time-to-resume	Mute rate; abandonment
Plain result vs. result + touched resources	Resource disclosure improves trust calibration	Correct recall of what changed	Cognitive load
Toast Undo vs. persistent history/ checkpoint	Durable recovery improves recovery for multi-step tasks	Successful recovery rate	Discovery time
Generic Confirm vs. object-specific commit CTA	Explicit action labels reduce accidental approvals	Post-confirmation reversal/regret	Completion latency
Narrated execution vs. state summaries	Summaries reduce cognitive load while preserving orientation	User comprehension of task status	Takeover latency
Agent personality vs. competence-oriented neutral UI	Reduced anthropomorphic framing improves calibrated trust in transactional tasks	Trust calibration / correctness judgment	Satisfaction and warmth
Default proactivity vs. user-selected proactivity level	User control improves long-term acceptance	30-day enabled retention	Missed-value rate

The proactive timing experiments are directly motivated by the 2025 controlled study, the 2026 field study and IUI 2026 work on assistance timing. ⁶

Metric definitions worth instrumenting

Verified task success should mean the intended target state actually exists, not that the agent generated a completion message. Agentic systems need outcome verification because execution can fail partially or produce persistent side effects that users do not notice. AgentTrace's research was explicitly motivated by the gap between high-level requests and opaque lower-level effects. ¹²

Track:

Metric	Definition
Verified task success	Successful intended state / attempted tasks
Unintended side-effect rate	Tasks with unauthorized or unintended material changes / executed tasks
Supervisory burden	User-active attention time during delegated work
Interventions per success	Clarifications + approvals + takeovers / successful tasks
Clarification efficiency	Useful information gained / clarification interruption
Approval regret	Undo/reversal shortly after approval
Recovery success	Failed/incorrect tasks restored to usable state without abandonment
Recovery time	Time from detected problem to restored usable state
Time to useful progress	Time from delegation to first verifiable useful result
Completion latency	Delegation to verified final state
Plan correction rate	Tasks where users modify generated plan
Takeover rate	Tasks requiring manual intervention
Proactive acceptance	Accepted proactive offers / visible proactive offers
Proactive suppression	Dismiss, snooze or mute events / suggestions
Trace inspection rate	Tasks where users inspect evidence/activity
Trust calibration	Relationship between user confidence and objective agent correctness
Accessibility parity gap	Difference in success/time/error rates across keyboard, screen-reader and other supported interaction modes

Do not automatically interpret a high Undo rate as failure. During early adoption, visible Undo may **increase willingness to delegate**. Look at whether undo follows genuine model error, preference correction, experimentation, or accidental user approval.

Likewise, a falling confirmation rate can mean either a beautifully calibrated system or users blindly approving things. Pair behavioral metrics with comprehension testing.

Designer's release checklist

Check	Ship criterion
☐ Object clarity	The primary Task, objects being acted upon and current state are visually identifiable.

Check	Ship criterion
☐ Goal visibility	The user's requested outcome remains inspectable during execution.
☐ Constraints	Important budget, time, privacy and scope constraints are visible and editable.
☐ Plan	Complex tasks expose a meaningful macro-plan; trivial ones are not forced through ceremony.
☐ State	Planning, Working, Waiting, Paused, Verifying, Done and Failed are distinguishable in text, not color alone.
☐ Control	Stop/Pause/Take over appear where and when they matter.
☐ Risk guard	Consequential external actions cross an explicit commit boundary.
☐ Confirmation copy	The button names the actual action/object rather than saying Yes/Continue.
☐ Reversibility	Reversible actions provide Undo, history or restore where technically feasible.
☐ Failure recovery	Every important failure has a next step beyond "Something went wrong."
☐ Traceability	Users can determine what resources were touched and what material changes occurred.
☐ AI provenance	AI-created or AI-modified content has a consistent, accessible provenance mechanism when provenance matters. IBM Carbon offers a useful reference pattern. ¹⁶
☐ Proactivity restraint	Suggestions have contextual value, sensible timing, dismissal and a way to reduce future interruptions.
☐ Background policy	Scheduled tasks expose next run, scope, conditions and pause/revoke controls.
☐ No hidden commitment	Investigation cannot silently become purchase/send/delete without the intended policy boundary.
☐ Accessibility	Pointer targets satisfy WCAG requirements; all agent controls are keyboard accessible; progress/errors are programmatically exposed. ⁷⁵
☐ Motion	Non-essential agent animation respects reduced-motion preferences. ⁷⁶
☐ Contrast	Text and meaningful UI graphics meet applicable WCAG contrast criteria. ⁶⁸
☐ Localization	Dynamic labels can expand, RTL works, and dates/currency/time zones are localized correctly. W3C specifically recommends avoiding constrained text containers for localization. ⁶⁹
☐ Evidence of completion	"Done" is backed by observable target-state verification rather than only an agent assertion.

Rules to give an AI model generating OOUX, flows and screens

For a design-generating model, I would make the following rules part of its persistent system or design-system context:

1. **Start from objects and states, not chat messages.** Identify Task, Goal, Constraints, Plan, Steps, Actions, Resources, Approvals, Outcomes and Events before drawing screens.
2. **For every agent action, classify consequence and reversibility.** Read-only, reversible local, external recoverable, consequential, destructive, or recurring/background.
3. **Every consequential action requires an explicit UX policy.** The model must specify whether it previews, confirms, executes with undo, or is prohibited.
4. **Every screen must expose the user's current agency.** At minimum identify what the user can edit, approve, interrupt, take over, undo or leave alone.
5. **Separate task state from conversational history.** Chat may create or modify task objects but should not be the only representation of persistent work.
6. **Show macro-level process, not hidden reasoning.** Use plans, steps, resources, evidence, diffs and outcomes.
7. **Treat error recovery as part of the happy path.** Generate failure, partial-success, stale-session, conflict, denied-permission, timeout and incorrect-result states alongside success.
8. **Prefer inline suggestions; reserve modal interruption for true decision boundaries.**
9. **Make proactivity contestable.** Every proactive pattern must define Dismiss, Snooze/Mute where appropriate, `Why this?`, and the rule for whether it can recur.
10. **Use verb + object action labels.** Never generate `Yes`, `OK`, `Continue`, or `Confirm` when a more specific consequential verb is available.
11. **Preserve object identity across turns.** Users should be able to say "keep the second flight," "undo the calendar change," or "change step 3" without reconstructing context.
12. **Design the mobile information architecture from the same object model, not by shrinking desktop layouts.**

Those rules synthesize the strongest patterns across PAIR, HAX, Carbon, Magentic-UI, AgentTrace, Replit and current production agent documentation. ⁷⁷

The north-star principle is ultimately straightforward:

Increase autonomy by reducing the cost of understanding, intervention, and recovery, not by hiding the agent's work.

The best agentic interface should feel less like supervising an unpredictable chatbot and more like delegating to a system whose contract, progress, boundaries and consequences remain legible. Research and current production patterns increasingly converge on that model: co-planning when complexity warrants it, sparse and well-timed initiative, visible execution state, risk-adjusted confirmation, contextual takeover, durable traceability, and first-class recovery. ⁷⁸

¹ ⁴ ⁷ ²³ ⁴⁷ ⁵⁹ <https://openai.com/index/introducing-chatgpt-agent/>

<https://openai.com/index/introducing-chatgpt-agent/>

² ¹⁵ ⁴⁹ ⁶⁰ ⁷⁷ <https://pair.withgoogle.com/chapter/mental-models/>

<https://pair.withgoogle.com/chapter/mental-models/>

³ ⁹ ¹⁰ ²² ⁵¹ ⁵⁵ ⁶¹ ⁷² ⁷⁸ <https://arxiv.org/html/2507.22358v1>

<https://arxiv.org/html/2507.22358v1>

⁵ ⁴¹ <https://docs.replit.com/learn/plan-vs-build-mode>

<https://docs.replit.com/learn/plan-vs-build-mode>

⁶ ¹¹ ⁵² <https://arxiv.org/html/2410.04596v2>

<https://arxiv.org/html/2410.04596v2>

⁸ ¹² ⁷⁴ <https://arxiv.org/html/2603.28551v2>

<https://arxiv.org/html/2603.28551v2>

¹³ ⁵⁴ <https://arxiv.org/html/2602.01481>

<https://arxiv.org/html/2602.01481>

¹⁴ <https://arxiv.org/abs/2601.10253>

<https://arxiv.org/abs/2601.10253>

¹⁶ <https://carbondesignsystem.com/guidelines/carbon-for-ai/>

<https://carbondesignsystem.com/guidelines/carbon-for-ai/>

¹⁷ <https://www.w3.org/TR/WCAG22/>

<https://www.w3.org/TR/WCAG22/>

¹⁸ <https://www.microsoft.com/en-us/haxtoolkit/design-patterns/>

<https://www.microsoft.com/en-us/haxtoolkit/design-patterns/>

¹⁹ ⁶⁷ <https://www.nngroup.com/articles/designing-ai-study-guide/>

<https://www.nngroup.com/articles/designing-ai-study-guide/>

²⁰ <https://developer.apple.com/design/human-interface-guidelines/generative-ai>

<https://developer.apple.com/design/human-interface-guidelines/generative-ai>

²¹ <https://arxiv.org/abs/2608.18638>

<https://arxiv.org/abs/2608.18638>

²⁴ ChatGPT agent

https://help.openai.com/en/articles/11752874-chatgpt-agent?utm_source=chatgpt.com

25 26 27 <https://www.anthropic.com/product/claude-cowork>

<https://www.anthropic.com/product/claude-cowork>

28 29 30 46 53 71 [https://support.google.com/gemini/answer/17094507?](https://support.google.com/gemini/answer/17094507?co=GENIE.Platform%3DDesktop&hl=en)

[co=GENIE.Platform%3DDesktop&hl=en](https://support.google.com/gemini/answer/17094507?co=GENIE.Platform%3DDesktop&hl=en)

<https://support.google.com/gemini/answer/17094507?co=GENIE.Platform%3DDesktop&hl=en>

31 33 34 <https://www.perplexity.ai/help-center/en/articles/12781449-comet-for-enterprise.html>

<https://www.perplexity.ai/help-center/en/articles/12781449-comet-for-enterprise.html>

32 <https://www.perplexity.ai/comet/resources/articles/comet-data-privacy-security-faq-s>

<https://www.perplexity.ai/comet/resources/articles/comet-data-privacy-security-faq-s>

35 <https://manus.im/features/manus-browser-operator>

<https://manus.im/features/manus-browser-operator>

36 38 <https://docs.replit.com/features/agent/overview>

<https://docs.replit.com/features/agent/overview>

37 <https://docs.replit.com/features/agent/plan-mode>

<https://docs.replit.com/features/agent/plan-mode>

39 40 48 63 <https://docs.replit.com/features/version-control/checkpoints-and-rollbacks>

<https://docs.replit.com/features/version-control/checkpoints-and-rollbacks>

42 43 73 <https://www.aboutamazon.com/news/retail/alexa-for-shopping-ai-assistant>

<https://www.aboutamazon.com/news/retail/alexa-for-shopping-ai-assistant>

44 <https://www.aboutamazon.com/news/retail/amazon-shopping-app-buy-for-me-brands>

<https://www.aboutamazon.com/news/retail/amazon-shopping-app-buy-for-me-brands>

45 62 <https://aws.amazon.com/blogs/machine-learning/how-rufus-scales-conversational-shopping-experiences-to-millions-of-amazon-customers-with-amazon-bedrock/>

<https://aws.amazon.com/blogs/machine-learning/how-rufus-scales-conversational-shopping-experiences-to-millions-of-amazon-customers-with-amazon-bedrock/>

50 <https://www.w3.org/TR/wcag2mobile-22/>

<https://www.w3.org/TR/wcag2mobile-22/>

56 <https://www.w3.org/WAI/WCAG21/Understanding/non-text-contrast.html>

<https://www.w3.org/WAI/WCAG21/Understanding/non-text-contrast.html>

57 75 <https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>

<https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>

58 <https://pair.withgoogle.com/chapter/errors-failing/>

<https://pair.withgoogle.com/chapter/errors-failing/>

64 <https://www.w3.org/WAI/WCAG21/Understanding/animation-from-interactions>

<https://www.w3.org/WAI/WCAG21/Understanding/animation-from-interactions>

65 <https://www.w3.org/WAI/WCAG22/Understanding/status-messages.html>

<https://www.w3.org/WAI/WCAG22/Understanding/status-messages.html>

66 <https://www.w3.org/WAI/WCAG22/Techniques/aria/ARIA19>

<https://www.w3.org/WAI/WCAG22/Techniques/aria/ARIA19>

⁶⁸ <https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum>
<https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum>

⁶⁹ <https://www.w3.org/International/techniques/authoring-html>
<https://www.w3.org/International/techniques/authoring-html>

⁷⁰ <https://www.w3.org/TR/string-meta/>
<https://www.w3.org/TR/string-meta/>

⁷⁶ <https://www.w3.org/WAI/WCAG22/Techniques/css/C39>
<https://www.w3.org/WAI/WCAG22/Techniques/css/C39>